Simulation of Data logger:-

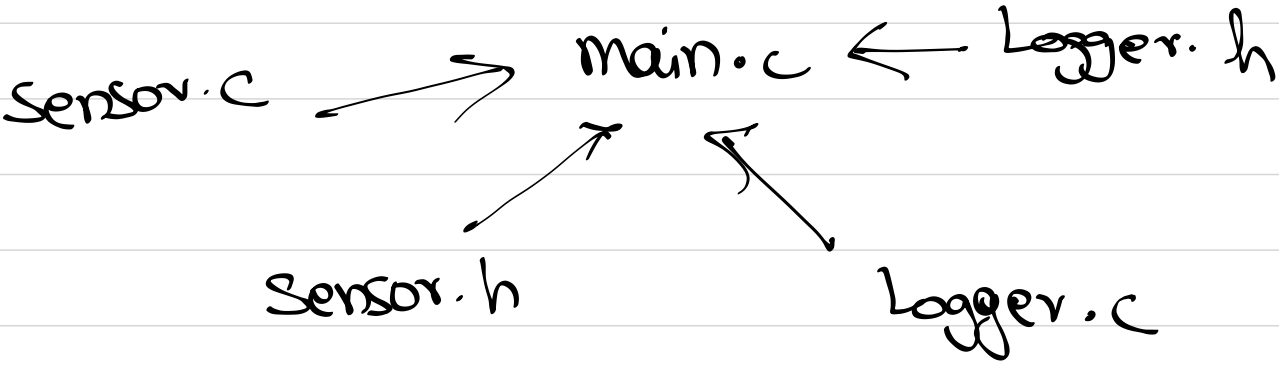
Used Flow:-

Simulates speed / temp → Allocate to struct → Save into CSV.

Real World scenario

Timer / Scheduler → Read Sensors → Process Data → log / transmit

Structure :



Sensor.h :

(if this file is not defined)

```
#ifndef SENSOR_H
#define SENSOR_H

typedef struct {
    int     timestamp;
    float   temperature;
    float   speed;
} SensorData;

SensorData generate_sensor_data(int timestamp);

#endif
```

then define this file (one time only)

Data structure defination

type of definition

Name of defined DS

temp & speed value w.r.t timestamp

End of condiditional definition of header file.

sensor.c:

stand. lib functions e.g rand,....

```
#include <stdlib.c>
#include "sensor.h"
```

header file

Declared in header

Sensor Data declaration

```
SensorData generate_sensor_data(int timestamp)
```

Function defination

```
{ SensorData data;
  data.timestamp = timestamp;

  data.temperature = 20.0f + (rand() % 1500)
                                  / 100.0f;
```

→ Local variable creation

floating point (32 bit)

to avoid integer divi.

```
  data.speed = (rand() % 2000) / 100.0f;

  return data;
```

Range limiter

(64 bit)

20.0 = double

20.0f = float

(32 bit)

Memory optimization

(Since we know precision requirements)

⇒ logger.h:

```
#ifndef LOGGER_H
#define LOGGER_H

#include "sensor.h"

void init_logger (const char * filename);
void log_data (SensorData data);
void close_logger ();

#endif
```

to be able to read sensor data

logger initialization

Logging the data

Closing the logger file.

⇒ Logger.c:

Makes the logger file private, no other .c can access it.

```
#include <stdio.h>
#include "Logger.h"

static FILE *file = NULL;

void init_logger (const char *filename)
{
    file = fopen(filename, "w");
    if (file == NULL) {
        printf("Error");
    }
}
```

FILE pointer from stdio.h.

initially NULL.

Logger file

opening file in write mode

if fail to open

```
fprintf (file, "timestamp, temp, speed\n");
```

↳ writing CSV header

```
}
```

Logging data now

```
void log_data (SensorData data)
{ if file != NULL ) {
    fprintf (file, "%d, %.2f, %.2f\n",
                data.timestamp,
                data.temperature,
                data.speed);
}
```

writing the log data

.2f => upto 2 decimal pts.

```
void close_logger()
{ if file != NULL) {
        fclose (file);
    }
}
```

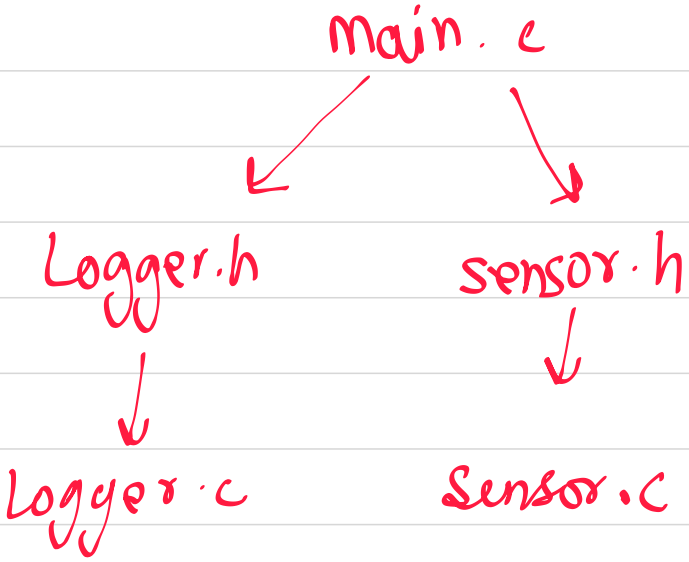
* Using "static" prevents from file misuse as it will contain important data for processing.

* != NULL is the defensive check.

Main.c :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include "sensor.h"
#include "logger.h"


int main()
{
    srand(time(NULL));

    init_logger("sensor_log.csv");

    for (int t=0; t<100; t++)

    {
        SensorData data = generate_sensor_data
                          (t);

        log_data(data);

        printf(" T=%d Temp=%.2f Speed
                =%.2f \n",
            data.timestamp,
            data.temperature,
```

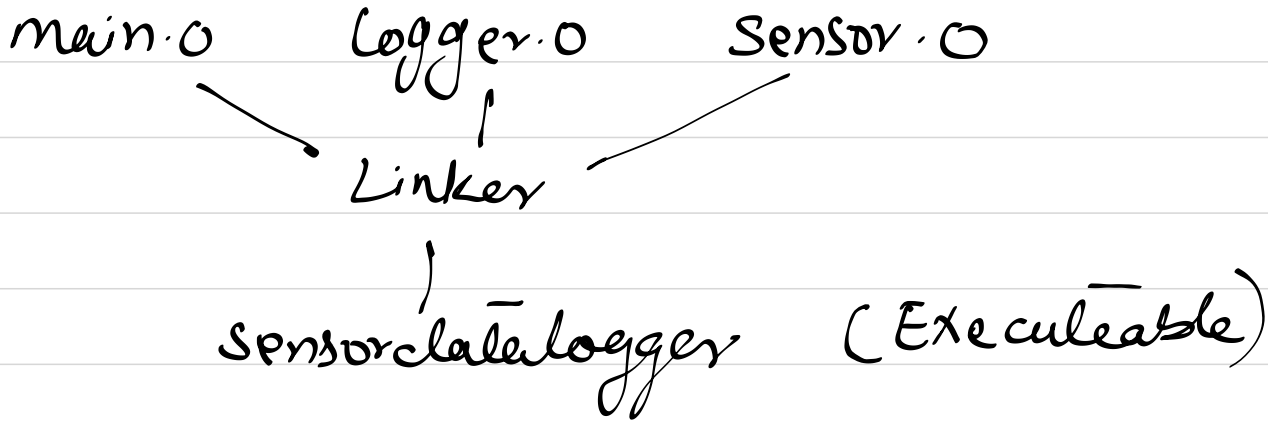
```
        delay.speed );
    }
    close_logger ();
    return 0;
}
```



- One compilation is done, re-compilation will happen for only changed .c files.

- Modularity is achieved in project by putting same type of functions togather in one .c file.

```
gcc main.c logger.c sensordata.c -o
    sensordatalogger
```

↑ linker step to produce executable "sensordatalogger" file

So, AS final compilation step: